WEEK 1 PROJECT

Data Scientist Intern — Setup & SQL Basics

A beginner-friendly, submission-ready guide based on your internship portal task

What the portal asks you to do

• Set up Python/Jupyter and a free SQL database.

• Write 15 SQL queries using SELECT, JOIN, and GROUP BY.

• Test your work, fix errors, and submit a short note describing what you built and learned.

Recommended simple route: Use Google Colab for Python (no installation required) and SQLite for a small free SQL database. If your mentor specifically requires Jupyter or PostgreSQL/Supabase, use the alternative instructions in this guide.

Deliverables to submit

• week1_sql_queries.sql — 15 working SQL queries

• week1_project_report.pdf or README.md — setup, dataset, query explanations, and learning note

• Screenshots showing the SQL queries/results and Python environment, if the portal allows attachments.

Important: The examples below use a small sample retail dataset created for practice. If your portal provides a required dataset, replace the sample data with that dataset and adjust table/column names. Do not claim you used a tool or dataset you did not actually use.

1. Setup — Python and SQL

Option A: Python without installation (easiest)

• Open https://colab.research.google.com/ and sign in with your Google account.

• Choose New notebook.

• Rename it Week1_SQL_Basics.ipynb.

• Run this cell to check Python:

```
print("Python setup is working!")
import pandas as pd
print("Pandas version:", pd.__version__)
```

If the cell runs and prints a message, your Python environment is ready. You can use SQLite from Python too:

```
import sqlite3
conn = sqlite3.connect("week1_retail.db")
print("SQLite database connected successfully")
```

Option B: Jupyter on your computer

```
python -m venv .venv
# Windows:
.venv\\Scripts\\activate
# macOS/Linux:
source .venv/bin/activate
python -m pip install jupyter pandas
jupyter notebook
```

If commands are unfamiliar, use Google Colab first; it avoids installation issues.

Create the practice database

Run the following SQL in a SQLite-compatible editor or through Python. The sample has customers, products, and orders.

```
CREATE TABLE customers (
    customer_id INTEGER PRIMARY KEY,
    customer_name TEXT NOT NULL,
    city TEXT NOT NULL
);

CREATE TABLE products (
    product_id INTEGER PRIMARY KEY,
    product_name TEXT NOT NULL,
    category TEXT NOT NULL,
    price REAL NOT NULL
);

CREATE TABLE orders (
    order_id INTEGER PRIMARY KEY,
    customer_id INTEGER NOT NULL,
    product_id INTEGER NOT NULL,
    quantity INTEGER NOT NULL,
    order_date TEXT NOT NULL,
    status TEXT NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id)
);

INSERT INTO customers VALUES
(1, 'Anu', 'Kadapa'),
(2, 'Ravi', 'Hyderabad'),
(3, 'Meena', 'Vijayawada'),
(4, 'Kiran', 'Kadapa');

INSERT INTO products VALUES
(101, 'Notebook', 'Stationery', 50),
(102, 'Pen Pack', 'Stationery', 30),
(103, 'Water Bottle', 'Accessories', 180),
(104, 'Backpack', 'Accessories', 650),
(105, 'Headphones', 'Electronics', 900);

INSERT INTO orders VALUES
```

```
(1001, 1, 101, 2, '2026-10-01', 'COMPLETED'),
(1002, 1, 103, 1, '2026-10-02', 'COMPLETED'),
(1003, 2, 105, 1, '2026-10-03', 'COMPLETED'),
(1004, 3, 102, 5, '2026-10-04', 'PENDING'),
(1005, 4, 104, 1, '2026-10-05', 'COMPLETED'),
(1006, 2, 101, 3, '2026-10-06', 'CANCELLED'),
(1007, 3, 103, 2, '2026-10-07', 'COMPLETED'),
(1008, 1, 105, 1, '2026-10-08', 'COMPLETED');
```

To execute SQL from the Python notebook, use this helper after creating the connection:

```
def run_sql(query):
    return pd.read_sql_query(query, conn)

run_sql("SELECT * FROM customers")
```

2. The 15 SQL queries

Run each query and check the output. These examples cover SELECT, filtering, sorting, aggregate functions, JOIN, and GROUP BY.

1. Display all customers

```
SELECT * FROM customers;
```

2. Select only customer names and cities

```
SELECT customer_name, city\nFROM customers;
```

3. Find customers from Kadapa

```
SELECT * FROM customers\nWHERE city = 'Kadapa';
```

4. Show products costing more than 100

```
SELECT product_name, price\nFROM products\nWHERE price > 100;
```

5. Sort products from highest to lowest price

```
SELECT product_name, price\nFROM products\nORDER BY price DESC;
```

6. Show the five most expensive products

```
SELECT product_name, price\nFROM products\nORDER BY price DESC\nLIMIT 5;
```

7. Count all customers

```
SELECT COUNT(*) AS total_customers\nFROM customers;
```

8. Find the average product price

```
SELECT AVG(price) AS average_price\nFROM products;
```

9. Calculate quantity sold for each product

```
SELECT product_id, SUM(quantity) AS total_quantity\nFROM orders\nWHERE status = 'COMPLETED'\nGROUP BY product_id;
```

10. Count orders by status

```
SELECT status, COUNT(*) AS order_count\nFROM orders\nGROUP BY status;
```

11. Join orders with customer names

```
SELECT o.order_id, c.customer_name, o.order_date, o.status\nFROM orders o\nJOIN customers c ON o.customer_id = c.custo
```

12. Join orders with product names and prices

```
SELECT o.order_id, p.product_name, p.price, o.quantity\nFROM orders o\nJOIN products p ON o.product_id = p.product_id;
```

13. Show completed orders with customer and product

```
SELECT o.order_id, c.customer_name, p.product_name, o.quantity, o.status\nFROM orders o\nJOIN customers c ON o.custome
```

14. Calculate completed sales by product category

```
SELECT p.category, SUM(p.price * o.quantity) AS total_sales\nFROM orders o\nJOIN products p ON o.product_id = p.produc
```

15. Find customers whose completed spending is over 500

```
SELECT c.customer_name, SUM(p.price * o.quantity) AS total_spending\nFROM orders o\nJOIN customers c ON o.customer_id
```

SQL notes: WHERE filters rows before grouping; GROUP BY creates groups; HAVING filters grouped results; JOIN connects related tables; SUM and AVG calculate summary values.

3. Run the SQL queries using Python

If you are using Google Colab, create the tables and sample rows using the SQL statements above through Python. Here is a complete working pattern for executing each query in SQLite:

```
import sqlite3
import pandas as pd

conn = sqlite3.connect("week1_retail.db")

# Example: execute a SELECT query and display the result
query = """
SELECT p.category, SUM(p.price * o.quantity) AS total_sales
FROM orders o
JOIN products p ON o.product_id = p.product_id
WHERE o.status = 'COMPLETED'
GROUP BY p.category
ORDER BY total_sales DESC;
"""

result = pd.read_sql_query(query, conn)
display(result)
```

For easiest execution, you can also use a browser-based SQLite editor. If you use a different SQL database, check whether it supports the same syntax. These queries use SQLite-compatible syntax.

4. Short project report — copy and edit after testing

Project title: Week 1 — Python Setup and SQL Basics

Objective: To set up a Python environment and practise SQL queries for retrieving, filtering, joining, and summarising data.

Tools: Python, Pandas, SQLite, and Google Colab or Jupyter Notebook. Keep only the tools you actually used.

Dataset: A sample retail dataset containing customers, products, and orders. The tables are connected using customer_id and product_id.

Work completed: Created relational tables, inserted sample records, and wrote 15 SQL queries using SELECT, WHERE, ORDER BY, LIMIT, COUNT, AVG, SUM, JOIN, GROUP BY, and HAVING.

Learning outcomes: Learned how to retrieve selected columns, filter records, sort data, join related tables, and calculate grouped summaries. Also practised checking query results and correcting errors.

Conclusion: This project provided beginner-level practice in SQL and Python data access. The queries can be adapted to a larger dataset for further analysis.

Before submitting: Run every query, add screenshots of actual outputs, replace any details that do not match your work, and confirm the portal's submission format. Do not submit untested claims.

5. Final checklist

• Python notebook opens and runs without errors.

• SQLite database contains the three tables and sample data.

• All 15 SQL queries run successfully.

• Save the SQL statements as week1_sql_queries.sql.

• Save the notebook as Week1_SQL_Basics.ipynb.

• Add screenshots of real query outputs to the report if requested.

• Write a short, honest note explaining what you built and learned.

• Upload the required files in My Submissions on the internship portal and check for a confirmation/status.

Time-saving plan: First get the database and 15 queries working. Then take screenshots and finish the report. If time is short, use Google Colab + SQLite rather than spending time troubleshooting a complex database installation.